

U-HAP: Unified Authorization over Heterogeneous Policies with Efficient Multi-Resource Verification in Kubernetes

Krittapak Jairak*, Singha Junchan*, Phisitphon Pruksorranan*, Somchart Fugkeaw*

*School of Information, Computer and Communication Technology (ICT)

Sirindhorn International Institute of Technology, Thammasat University, Thailand

6622782207@gsiit.tu.ac.th, 6622770350@gsiit.tu.ac.th, 6622782322@gsiit.tu.ac.th, somchart@gsiit.tu.ac.th

Abstract—Kubernetes environments increasingly host multiple applications, each governed by independently defined authorization policies such as RBAC, ABAC, and ACL. This heterogeneity results in repeated and fragmented authorization checks, increased latency, inconsistent decisions, and management complexity. While Single Sign-On (SSO) addresses authentication, it does not unify authorization across heterogeneous policies. We propose U-HAP, a compilation-driven authorization framework that integrates diverse policy models within a unified graph-based semantic representation and compiles them into resource–action scoped indices for efficient request-time evaluation. U-HAP introduces a two-level pruning mechanism that restricts evaluation to only the policies relevant to each request, ensuring deterministic conflict resolution without redundant checks. We implement U-HAP as a Kubernetes webhook authorizer and evaluate it against a traditional sequential evaluator under controlled in-process conditions. Experimental results demonstrate $O(1)$ namespace isolation across 1,000 applications, $3.0\text{--}3.2\times$ algorithmic speedup through compiled evaluation, up to $15.9\times$ speedup with decision caching, and up to $43.4\times$ speedup for ABAC-intensive workloads via hash-consed sub-expression memoization. For pure RBAC workloads without ABAC predicates, U-HAP incurs a $0.3\text{--}0.4\times$ overhead due to bit-vector construction cost.

Index Terms—access control, Kubernetes, policy unification, directed acyclic graph, hash consing, RBAC, ABAC, ACL, webhook authorization, Zero Trust

I. INTRODUCTION

Kubernetes is the de facto platform for orchestrating containerized applications, including security-critical systems [1]. Yet authorization remains a major challenge: misconfigurations and excessive permissions frequently lead to vulnerabilities and full cluster compromise [2], [3].

Authorization in Kubernetes is inherently *multi-application, multi-resource*. A single workflow may touch multiple pods, namespaces, or clusters, each governed by independently defined policies (RBAC [4], ABAC, ACL [5], or custom) that operate under incompatible semantics. This results in repeated checks, inconsistent decisions, and operational inefficiency, even when SSO reduces authentication overhead.

Existing authorization mechanisms suffer from several fundamental limitations. First, semantic fragmentation arises because widely adopted policy frameworks such as RBAC, ABAC, and ACLs employ incompatible policy languages and evaluation semantics, leading to increased policy duplication and a higher risk of misconfiguration [6]. Second,

conflicts among multiple authorization mechanisms are implicitly resolved based on evaluation order, resulting in non-deterministic and potentially inconsistent access decisions. Third, runtime inefficiency persists due to reliance on repeated policy scanning, even when only a small subset of rules is relevant to a given request, thereby introducing unnecessary computational overhead. Although existing solutions attempt to alleviate some of these challenges, they remain insufficient in supporting efficient and scalable *multi-model authorization* across diverse systems and policies [8], [9].

We propose **U-HAP**, a *compilation-driven, index-based* authorization framework designed to address the inefficiencies and inconsistencies of heterogeneous policy enforcement in modern cloud-native systems. The core idea is to shift the computational burden from runtime to an offline compilation phase, where diverse policies are transformed into resource–action scoped indices. This enables an efficient *two-level pruning mechanism*: first, *policy-type pruning* activates only the relevant classes of policies (e.g., RBAC, ABAC, ACL, or deny rules) for a given request; second, *token-driven pruning* further restricts evaluation to rules that match the user’s roles, groups, and attributes. By combining these pruning strategies with a unified graph-based abstraction, U-HAP ensures formally consistent and deterministic authorization decisions, while significantly reducing runtime overhead. Although our prototype is implemented as a Kubernetes webhook authorizer, the underlying design is general and can be readily extended to other distributed and multi-application environments.

This paper makes the following key contributions. First, we formalize the problem of *multi-model multi-resource authorization* and introduce a unified graph-based semantic model that seamlessly integrates diverse policy paradigms, including RBAC, ABAC, ACL, and explicit deny rules, within a single coherent framework. Second, we develop a *compilation-driven authorization architecture* that precomputes resource–action indices, thereby eliminating the need for costly runtime policy scanning and enabling constant-time access to relevant policy artifacts. Third, we design a lightweight yet effective *two-level pruning strategy* that minimizes verification cost by restricting evaluation to only the subset of policies that are both structurally and contextually relevant to each request. Finally,

we implement and evaluate U-HAP as a Kubernetes webhook authorizer, demonstrating that it achieves efficient, low-latency, and deterministic authorization with unified decision semantics and minimal operational overhead.

II. RELATED WORK

Security and access control in Kubernetes environments have been widely studied, particularly from the perspectives of system hardening and configuration correctness. Foundational guidelines such as the NSA/CISA Kubernetes Hardening Guidance [1] and subsequent compliance-oriented analyses [7] emphasize secure control plane configuration, workload isolation, and least-privilege enforcement. However, empirical studies reveal that misconfigurations and excessive permissions remain pervasive in real-world deployments [2], [3], often enabling privilege escalation and even full cluster compromise. Recent tools such as EPScan [10] further confirm that excessive RBAC permissions in third-party applications continue to be a systematic security risk. Complementary work on formal verification [16] demonstrates that static policy analysis can prevent misconfigurations in Kubernetes clusters, yet such approaches address correctness of individual policies rather than unified authorization across mixed-model environments.

Kubernetes natively adopts Role-Based Access Control (RBAC) [4], which provides scalable permission management but lacks expressiveness for fine-grained, context-aware authorization. To address these limitations, policy-as-code frameworks such as Open Policy Agent (OPA) [18] and Kyverno [19] enable declarative and dynamic policy enforcement, while Zero Trust architectures [11] promote continuous verification and context-aware access decisions in distributed cloud-native environments. Nevertheless, these approaches operate on independently evaluated policy models and require repeated evaluation across multiple enforcement layers, leading to redundant checks and increased latency in multi-resource authorization scenarios [6]. Recent optimization efforts such as PerfSPEC [12] improve enforcement efficiency through performance-aware profiling, and compile-time policy optimization techniques [17] reduce redundant rule evaluation. However, both approaches operate within individual policy models and do not address unified authorization across diverse policy types.

Beyond RBAC, Attribute-Based Access Control (ABAC) and standardized frameworks such as XACML [8] provide more expressive and fine-grained authorization capabilities. However, XACML relies on heavyweight policy decision point (PDP) architectures that are not well suited for lightweight, low-latency cloud-native systems. Prior work on compiling XACML policies into indexed evaluation structures [20] has demonstrated significant speedups over sequential evaluation, motivating the compile-time optimization approach adopted by U-HAP. Recent works explore graph-based representations of access control, modeling authorization as relationship inference over entities and attributes [9], [13]. Large-scale systems such as Zanzibar [14] demonstrate the scalability of relationship-based authorization, but require runtime graph

traversal, introducing overhead proportional to graph complexity. Similarly, approaches that convert ABAC policies into RBAC models [15] improve interoperability but do not resolve the challenge of unified and efficient runtime authorization across diverse policy types.

Overall, existing works either focus on hardening and misconfiguration detection, enhancing policy expressiveness, or improving verification and enforcement mechanisms. However, they do not provide a unified authorization framework that efficiently supports diverse policy models (e.g., RBAC, ABAC, ACL, and deny rules) while minimizing redundant evaluation and ensuring scalable, low-latency multi-resource authorization in cloud-native environments.

III. PROPOSED SYSTEM

A. System Model

U-HAP separates *policy compilation* (offline) from *request-time authorization* (online) to achieve low-latency, deterministic enforcement (Fig. 1).

System Entities.

- **User ($\mathcal{U}$):** Authenticated subject issuing requests with dynamic attributes (roles, groups, context).
- **Administrator:** Defines policies (RBAC, ABAC, ACL, Deny) via the U-HAP DSL.
- **Kubernetes API Server ($\mathcal{K}$):** Dispatches authorization queries via `SubjectAccessReview` (SAR).
- **U-HAP Webhook ($\mathcal{W}$):** Evaluates SAR requests using compiled artifacts and returns decisions.
- **DSL Loader ($\mathcal{L}$):** Parses and validates policies.
- **Policy Compiler ($\mathcal{B}$):** Compiles policies into resource-action scoped artifacts $\{\mathcal{C}_{n,r,a}\}$.
- **Graph Registry ($\mathcal{M}$):** In-memory store for compiled artifacts, enabling $O(1)$ lookup.
- **Resources ($\mathcal{O}$):** Protected Kubernetes objects organized by namespace.

U-HAP decouples authentication from authorization by deferring user-specific bindings to request time rather than embedding them in precomputed structures. Identity, roles, and attributes are supplied via tokens at request time, enabling dynamic evaluation without recompilation.

Trust Assumptions. The API server and U-HAP components are trusted. Policies are assumed correct after validation. Adversaries may issue malicious requests but cannot tamper with compiled artifacts or the registry.

B. Formal Graph Model

U-HAP defines a unified graph abstraction for heterogeneous authorization policies. For each namespace-resource pair (n, r) , the system constructs

$$G_{n,r} = (V_{n,r}, E_{n,r}) \quad (1)$$

where $V_{n,r}$ and $E_{n,r}$ capture all policy semantics for resource r .

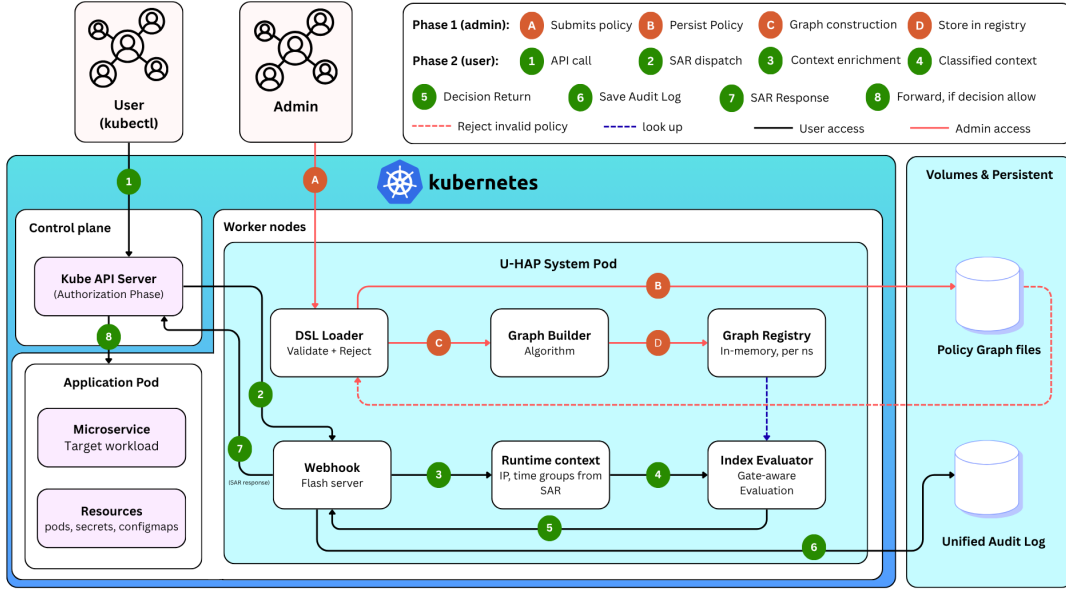


Fig. 1. U-HAP compilation-driven architecture on Kubernetes. **Phase 1 — Setup** (admin path, steps A–D): policy ingestion, validation, graph construction, and registry storage. **Phase 2 — Compilation & Request Dispatch** (user path, steps 1–4): SAR dispatch, context enrichment, index-based artifact lookup $C_{n,r,a}$, and decision evaluation. **Phase 3 — Evaluation** (steps 5–8): deny check, ACL/RBAC/ABAC index evaluation, cache update, and SAR response.

The vertex set $V_{n,r}$ comprises typed nodes: subject nodes $s \in \mathcal{S}$ (authenticated identities), role nodes $\rho \in \mathcal{R}$ (RBAC abstractions), gate nodes $g \in \mathcal{G}$ (ABAC predicates), deny nodes $d \in \mathcal{D}$ (explicit denials), and a distinguished resource node r . The edge set $E_{n,r}$ encodes relationships: (s, ρ) for subject–role assignments, (ρ_i, ρ_j) for role hierarchy, (ρ, r) for role-based permissions, (s, r) for direct ACL permissions, (s, g) and (g, r) for attribute-based constraints, and (s, d) , (d, r) for explicit deny semantics.

Well-formedness requires that (i) the role hierarchy is acyclic, (ii) all edges terminate at a gate, deny, or resource node, and (iii) deny nodes have highest precedence, ensuring deterministic conflict resolution.

Authorization is defined as a constrained reachability problem over $G_{n,r}$. Given a request (s, a, r) with context $\mathbf{x}$, access is denied if a path $s \rightarrow d \rightarrow r$ exists; otherwise, access is granted if: (i) a direct edge (s, r) exists (ACL), (ii) a path $s \rightarrow \rho^* \rightarrow r$ exists through the role hierarchy (RBAC), or (iii) a path $s \rightarrow g \rightarrow r$ exists such that $g(\mathbf{x})$ is true (ABAC). If none apply, access is denied by default.

Critically, the graph is *not* traversed at runtime; it is compiled in Phase 2 into resource–action scoped artifacts $C_{n,r,a}$, and Phase 3 performs index-based evaluation with policy-type and token-driven pruning, ensuring correctness without graph traversal.

Phase 1: Setup: The setup phase initializes the U-HAP environment and is performed once during deployment.

System Initialization. Let $\mathcal{N}$ denote namespaces and $\mathcal{O}_n$ the protected resources in namespace $n \in \mathcal{N}$. The Kubernetes API server delegates authorization via SAR requests to the U-HAP

Webhook. Each request is normalized as:

$$\mathcal{Q} = \langle \tau, (n, r, a) \rangle \quad (2)$$

Component Deployment. U-HAP deploys four core components: (i) the DSL Loader, (ii) the Policy Compiler/Graph Builder, (iii) the Graph Registry, and (iv) the Authorization Webhook. The first two operate offline; the latter two serve runtime authorization.

State Initialization. The system initializes a *persistent policy store* (e.g., ConfigMaps) and an *in-memory registry*. For each (n, r, a) , the registry stores:

$$C_{n,r,a} = (G_{n,r}, I_{n,r,a}^{acl}, I_{n,r,a}^{rbac}, I_{n,r,a}^{abac}, I_{n,r,a}^{attr}, I_{n,r,a}^{deny}, \Psi_{n,r,a}) \quad (3)$$

populated during Phase 2 (individual artifacts are defined in Table I).

Token Trust Configuration. Each validated token is modeled as:

$$\tau = \langle uid, \mathcal{R}_u, \mathcal{G}_u, \mathcal{A}_u, t_{exp}, \sigma \rangle \quad (4)$$

The token is assumed authentic before entering authorization. U-HAP supports policy types $\Pi_n = \{\text{RBAC}, \text{ABAC}, \text{ACL}, \text{Deny}\}$ with predefined schema and constraints.

At the end of setup, U-HAP establishes: (i) a unified request model, (ii) trusted token validation, (iii) a persistent policy store, and (iv) a structured registry for compiled artifacts—forming the foundation for deterministic compilation (Phase 2) and efficient authorization (Phase 3).

Phase 2: Policy Compilation. The compilation phase transforms DSL policies into *resource-action-scoped runtime artifacts* consumed by the verifier in Phase 3. Rather than relying on runtime graph traversal, U-HAP compiles policies into verification-oriented indices enabling constant-time lookup, token-driven pruning, and short-circuit evaluation.

Input and Output. For $\Pi_n = \{\text{RBAC}, \text{ABAC}, \text{ACL}, \text{Deny}\}$, the compiler produces:

$$\mathcal{C}_n = \{\mathcal{C}_{n,r,a} \mid r \in \mathcal{O}_n, a \in \mathcal{A}\} \quad (5)$$

where each unit $\mathcal{C}_{n,r,a}$ aligns with the verification input $\langle \tau, (n, r, a) \rangle$, eliminating runtime filtering.

Compilation Procedure.

Step 1: Ingestion and IR Construction. Policies are parsed into:

$$\mathcal{IR} = (\mathcal{S}, \mathcal{R}, \mathcal{P}, \mathcal{A}, \mathcal{D}) \quad (6)$$

Step 2: Validation and Normalization. The DSL Loader enforces syntax, type safety, reference consistency, and acyclic role hierarchies. Role inheritance is reduced to transitive closure $\rho_i \Rightarrow^* \rho_j$.

Step 3: Resource-Action Decomposition. Policies are partitioned as $\Pi_n \rightarrow \{\Pi_{n,r,a}\}$, ensuring each artifact corresponds exactly to (n, r, a) .

Step 4: Artifact Generation. For each (n, r, a) , the artifacts shown in Table I are constructed.

TABLE I
COMPILED ARTIFACTS PER RESOURCE-ACTION SCOPE (n, r, a) .

Artifact	Definition	Notes
Graph	$G_{n,r}$	audit only
Deny	$I_{n,r,a}^{\text{deny}} = \{\text{scope}(d), \phi_d\}$	constant-time match
ACL	$I_{n,r,a}^{\text{acl}} = \mathcal{S}_{r,a}^{\text{acl}}$	membership check
RBAC	$I_{n,r,a}^{\text{rbac}} = \{\rho \mid \rho \Rightarrow^* (r, a)\}$	$\mathcal{R}_u \cap I_{n,r,a}^{\text{rbac}} \neq \emptyset$
ABAC	$I_{n,r,a}^{\text{abac}} = \{(g_\phi, \text{req}(\phi))\}$	$g_\phi : \mathcal{A}_u \rightarrow \{0, 1\}$
Attr	$I_{n,r,a}^{\text{attr}} : k \mapsto \{\phi \mid k \in \text{req}(\phi)\}$	selective eval.
Summary	$\Psi_{n,r,a} = \langle \mathcal{S}_{r,a}^{\text{acl}}, \mathcal{R}_{r,a}^{\text{rbac}}, \Phi_{r,a}^{\text{abac}}, \mathcal{D}_{r,a} \rangle$	–

Each ABAC predicate ϕ is compiled into a DAG of GateNode objects via *hash consing*: structurally identical sub-expressions across policies share a single node, so any predicate appearing in k policies maps to exactly one GateNode in the shared DAG.

Steps 5–6: Finalization and Registration. Redundant edges are removed, unreachable nodes are pruned, and consistency is verified. All $\mathcal{C}_{n,r,a}$ are then stored in the Graph Registry for $O(1)$ retrieval. ACL reduces to membership checking, RBAC to set intersection, ABAC to attribute-key-pruned predicate evaluation, and deny to prioritized filtering, eliminating global policy scans at runtime.

Phase 3: Request-Time Authorization. At runtime, U-HAP evaluates each request via a *policy-type-pruned, token-projected, index-optimized* procedure, operating only on compiled (n, r, a) -scoped artifacts from Phase 2.

Input. Each request $\mathcal{Q} = \langle \tau, (n, r, a) \rangle$ carries a validated token $\tau = \langle \text{uid}, \mathcal{R}_u, \mathcal{G}_u, \mathcal{A}_u, t_{\text{exp}}, \sigma \rangle$, where uid is the authenticated identity, $\mathcal{R}_u$ the role set, $\mathcal{G}_u$ the group set, and $\mathcal{A}_u$ the user and contextual attributes. Token validation is completed before authorization begins.

Artifact Retrieval. Given (n, r, a) , the Webhook performs an $O(1)$ lookup to retrieve $\mathcal{C}_{n,r,a}$, including the fast-path descriptor:

$$\Xi_{n,r,a} = \langle \lambda^{\text{deny}}, \lambda^{\text{acl}}, \lambda^{\text{rbac}}, \lambda^{\text{abac}} \rangle \quad (7)$$

Each activation bit $\lambda^* \in \{0, 1\}$ indicates whether the corresponding policy type has at least one compiled rule for (n, r, a) ; types with $\lambda = 0$ are skipped entirely, avoiding unnecessary evaluation.

Two-Level Pruning. U-HAP prunes candidates in two stages: *policy-type pruning* via $\Xi_{n,r,a}$ eliminates inactive classes, and *token-driven pruning* derives only candidates relevant to the current token:

$$\Gamma_{\text{deny}} = \{d \mid \text{scope}(d) \cap (\{\text{uid}\} \cup \mathcal{G}_u \cup \mathcal{R}_u) \neq \emptyset\} \quad (8)$$

$$\Gamma_{\text{acl}} = (\{\text{uid}\} \cup \mathcal{G}_u) \cap I_{n,r,a}^{\text{acl}} \quad (9)$$

$$\Gamma_{\text{rbac}} = \mathcal{R}_u \cap I_{n,r,a}^{\text{rbac}} \quad (10)$$

$$\Gamma_{\text{abac}} = \bigcup_{k \in \text{dom}(\mathcal{A}_u)} I_{n,r,a}^{\text{attr}}(k) \quad (11)$$

Decision Cache. Before rule evaluation, the verifier checks a short-lived cache indexed by

$$\kappa = H(\text{uid}, n, r, a, \mathbf{b}_u, \mathbf{g}_u, \text{sig}(\mathcal{A}_u)) \quad (12)$$

returning the cached decision immediately on hit. Cache entries are invalidated upon policy updates or token expiry.

Verification Order and Short-Circuit Logic. On cache miss, authorization follows strict precedence:

$$\text{Deny} \rightarrow \text{ACL} \rightarrow \text{RBAC} \rightarrow \text{ABAC} \rightarrow \text{Default Deny} \quad (13)$$

Deny Verification. If $\lambda^{\text{deny}} = 1$, only candidates in Γ_{deny} are checked. Any match immediately rejects the request:

$$\exists d \in \Gamma_{\text{deny}} : \text{match}_{\text{deny}}(d, \tau, n, r, a) = 1 \Rightarrow \text{DENY} \quad (14)$$

ACL Verification. If no deny rule applies and $\lambda^{\text{acl}} = 1$, direct permission is checked via constant-time hash membership:

$$\text{uid} \in I_{n,r,a}^{\text{acl}} \vee \mathcal{G}_u \cap I_{n,r,a}^{\text{acl}} \neq \emptyset \quad (15)$$

RBAC Verification via Bit-Vector Matching. If $\lambda^{\text{rbac}} = 1$, let $\mathbf{b}_u$ encode the user role set and $\mathbf{b}_{n,r,a}^{\text{rbac}}$ the precompiled authorized roles after transitive closure. RBAC authorization holds iff

$$\mathbf{b}_u \wedge \mathbf{b}_{n,r,a}^{\text{rbac}} \neq 0 \quad (16)$$

avoiding online hierarchy traversal, especially under large role universes.

ABAC Verification via Bucketed and Ordered Gates. Each predicate ϕ is compiled into a gate $g_\phi : \mathcal{A}_u \rightarrow \{0, 1\}$ with required keys $\text{req}(\phi)$ and an estimated cost. The attribute-key

index selects only predicates whose keys are present in the token, producing Γ_{abac} , then organizes them into lightweight buckets. Predicates are evaluated in ascending cost order:

$$\Gamma_{\text{abac}}^\uparrow = \text{sort}(\Gamma_{\text{abac}}, \text{cost}) \quad (17)$$

If any gate returns 1, the request is authorized:

$$\exists \phi \in \Gamma_{\text{abac}}^\uparrow : g_\phi(\mathcal{A}_u) = 1 \Rightarrow \text{ALLOW} \quad (18)$$

Gate evaluation remains compositional:

$$g_{\phi_1 \wedge \phi_2}(\mathcal{A}_u) = g_{\phi_1}(\mathcal{A}_u) \wedge g_{\phi_2}(\mathcal{A}_u) \quad (19)$$

$$g_{\phi_1 \vee \phi_2}(\mathcal{A}_u) = g_{\phi_1}(\mathcal{A}_u) \vee g_{\phi_2}(\mathcal{A}_u) \quad (20)$$

$$g_{\text{th}(k, \{\phi_i\})}(\mathcal{A}_u) = \mathbb{1} \left[\sum_i g_{\phi_i}(\mathcal{A}_u) \geq k \right] \quad (21)$$

Each GateNode is evaluated at most once per request via memoization, reducing effective ABAC cost from $O(|\text{policies}|)$ to $O(|\text{unique DAG nodes}|)$ and producing the sub-linear growth observed in Section IV, Fig. 3.

Default Deny and Cache Update. If no stage authorizes the request, the verifier returns DENY and inserts the decision into the cache under key κ .

Decision Function.

Verify(τ, n, r, a) =

$$\begin{cases} \text{Cache}[\kappa], & \text{if } \kappa \text{ hits} \\ \text{DENY}, & \exists d \in \Gamma_{\text{deny}} : \text{match}_{\text{deny}}(d, \tau, n, r, a) = 1 \\ \text{ALLOW}, & \text{uid} \in I_{n,r,a}^{\text{acl}} \vee \mathcal{G}_u \cap I_{n,r,a}^{\text{acl}} \neq \emptyset \\ \text{ALLOW}, & \mathbf{b}_u \wedge \mathbf{b}_{n,r,a}^{\text{rbac}} \neq \mathbf{0} \\ \text{ALLOW}, & \exists \phi \in \Gamma_{\text{abac}}^\uparrow : g_\phi(\mathcal{A}_u) = 1 \\ \text{DENY}, & \text{otherwise.} \end{cases} \quad (22)$$

Complexity. Let $d_{r,a} = |\Gamma_{\text{deny}}|$ and $w_{r,a} = |\Gamma_{\text{abac}}^\uparrow|$. In the cache-hit case, verification is $O(1)$. Otherwise:

$$\mathcal{O}(d_{r,a} + 1 + 1 + w_{r,a}) \quad (23)$$

where ACL reduces to constant-time hash membership, RBAC to one bitwise AND and zero-test, and ABAC evaluates only bucket-selected, attribute-relevant predicates in cost order.

Phase 3 thus achieves low-latency authorization through five complementary optimizations: (n, r, a) -scoped lookup, policy-type pruning, token-driven pruning, bit-vector RBAC matching, and bucketed ordered ABAC evaluation, further accelerated by short-lived decision caching.

IV. EXPERIMENTAL EVALUATION

This section evaluates U-HAP's runtime authorization performance. Two experiments are designed to validate the following claims: (1) U-HAP's compiled registry provides $O(1)$ namespace isolation, ensuring that authorization latency for a single application is independent of the total number of deployed applications; (2) U-HAP's decision cache provides

additional speedup for repeated requests; and (3) the per-model performance benefit of each compilation technique (hash consing for ABAC, hash-set lookup for ACL, bit-vector matching for RBAC) can be isolated and quantified.

A. Experimental Setup

All experiments run on the implemented U-HAP system (Python 3.11) on a machine with an AMD Ryzen 9 7945HX CPU (16 cores, 32 threads), 32 GB DDR5-4800 RAM, running CachyOS Linux. Evaluation is single-threaded per request, reflecting the Kubernetes API server's serialized SAR dispatch. Each data point is the median of 1,000 timed iterations after 50 warm-up iterations; timing uses `time.perf_counter_ns()` for nanosecond precision.

Baseline. To isolate the algorithmic contribution of U-HAP's compilation-driven evaluation from external factors such as network latency and runtime differences, we compare against a *traditional sequential evaluator* implemented in the same language (Python) and executed in the same process. The traditional evaluator represents a conventional authorization approach that stores policies as flat lists and evaluates them sequentially without pre-compilation:

- **Deny:** linear scan through all deny rules.
- **ACL:** linear scan through subject entries; $O(k)$.
- **RBAC:** for each RBAC rule, recursive hierarchy traversal to collect inherited roles, then linear scan; $O(k \cdot h)$ where h is hierarchy depth.
- **ABAC:** linear scan through all ABAC rules; for each rule, full predicate tree evaluation with no sub-expression memoization; $O(k \cdot g)$ where g is the number of gates per predicate.

This in-process comparison eliminates HTTP overhead, serialization cost, and language-runtime differences, ensuring that measured differences reflect purely algorithmic gains from U-HAP's compilation.

Policy Configuration. Each application namespace contains 46 heterogeneous authorization rules: 10 RBAC, 20 ABAC, 10 ACL, 1 deny, and 5 role-hierarchy edges. ABAC predicates use compound expressions with 10 gates (AND, OR, ATLEAST) and 11 attributes per rule, drawn from a pool of 26 attribute keys representative of organizational, location, security, and temporal categories. Approximately 50% of ABAC atoms are shared across rules within each namespace. Test requests are constructed to force worst-case ABAC evaluation: the probe user is not present in any ACL entry and holds no RBAC role, so the evaluator must scan all deny, ACL, and RBAC rules (all miss) before evaluating all ABAC predicates, with only the last ABAC rule matching. This ensures that measured latency reflects the full evaluation path rather than an early exit. In typical deployments, early short-circuiting (e.g., a deny or ACL match) would yield lower latency; the reported numbers therefore represent a conservative upper bound.

Compilation Cost. Phase 2 compilation is a one-time offline cost, amortized over all subsequent requests. For the standard

46-rule configuration, compilation completes in 9.9 ms (median, 100 runs). Even for a stress-test configuration of 500 ABAC rules with 10 gates each, compilation takes 187 ms—negligible relative to the thousands of authorization decisions served per second thereafter.

B. Namespace Isolation and Cache Effectiveness

This experiment evaluates whether U-HAP’s compiled registry provides $O(1)$ namespace isolation as the system scales, and quantifies the additional benefit of decision caching.

Setup. The total number of application namespaces N is varied from 10 to 1,000, with each namespace containing the standard 46-rule policy configuration described above. A single worst-case probe request is issued against a fixed target namespace (application 0), and only the Phase 3 evaluation time is measured. Three configurations are compared: (1) the traditional sequential evaluator (no compilation); (2) U-HAP with the decision cache disabled; and (3) U-HAP with the decision cache enabled and pre-warmed (all timed iterations are cache hits).

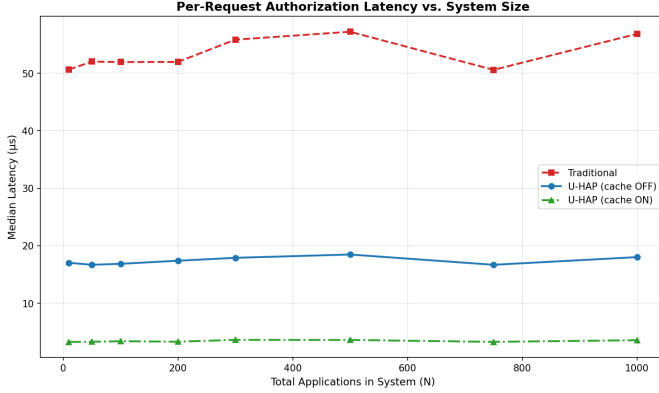


Fig. 2. Per-request authorization latency vs. total number of application namespaces. All three configurations exhibit flat latency, confirming $O(1)$ namespace isolation. U-HAP without cache achieves $3.0\text{--}3.2\times$ speedup over the traditional evaluator; with cache enabled, speedup reaches $15.3\text{--}15.9\times$.

Results. Fig. 2 shows that all three configurations maintain constant latency as N increases from 10 to 1,000. The traditional evaluator operates at $50.6\text{--}57.2\ \mu\text{s}$ per request (p95: $60.7\ \mu\text{s}$), U-HAP without cache at $16.7\text{--}18.5\ \mu\text{s}$ (p95: $18.5\ \mu\text{s}$; $3.0\text{--}3.2\times$ speedup), and U-HAP with cache at $3.3\text{--}3.6\ \mu\text{s}$ (p95: $3.6\ \mu\text{s}$; $15.3\text{--}15.9\times$ speedup). The narrow gap between median and p95 confirms low variance. No statistically significant trend is observed in any configuration as N grows.

Analysis. The flat-line behavior confirms that both systems index policies by namespace, so adding applications does not affect the evaluation cost of any individual application. The constant $3\times$ gap between the traditional evaluator and U-HAP (cache off) reflects the algorithmic advantage of compiled evaluation: hash-consed ABAC predicates are evaluated via memoized DAG traversal rather than redundant per-rule predicate evaluation, and ACL checks use hash-set membership rather than linear scans. The additional $5\times$ speedup from

caching demonstrates that for repeated identical requests—common in Kubernetes environments where the same user accesses the same resource multiple times within a short window—the decision cache effectively bypasses all evaluation logic. Together, these results validate U-HAP’s $O(1)$ namespace isolation and demonstrate that compilation and caching provide complementary, multiplicative performance benefits.

C. Per-Model Authorization Performance

This experiment isolates the performance contribution of each policy model (RBAC, ABAC, ACL) by evaluating workloads containing only one policy type at a time. This decomposition identifies which compilation technique contributes most to U-HAP’s overall speedup.

Setup. For each policy model, the number of rules k is varied from 10 to 500. In each configuration, only rules of the target type are present in the namespace. ABAC rules use 10 gates per predicate with 50% cross-rule atom sharing. The test request is designed as worst case for each model: the matching rule is placed last, so both evaluators must scan all k rules before finding a match. The decision cache is disabled to measure raw evaluation cost.

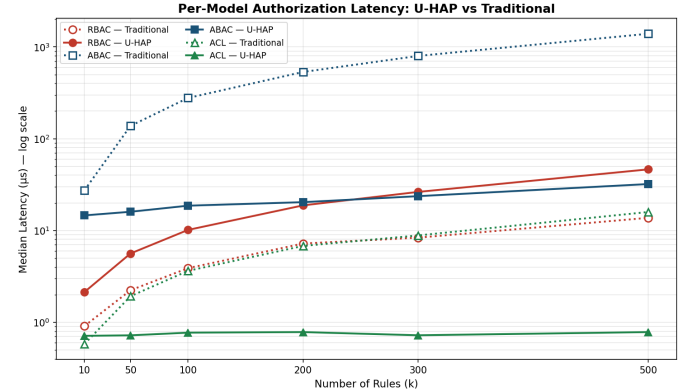


Fig. 3. Per-model authorization latency (log scale) vs. number of rules ($k=10\text{--}500$). Each access-control model is shown in a distinct color; solid lines denote U-HAP and dotted lines denote the traditional evaluator. At small k , compilation overhead dominates; the crossover occurs near $k=50$. At $k=500$: ABAC $43.4\times$, ACL $20.3\times$, RBAC $0.3\times$.

Results. Fig. 3 presents per-model results on a logarithmic scale. At small scale ($k=10$), compilation overhead dominates: ABAC achieves only $1.9\times$ speedup, ACL operates at $0.8\times$ (slightly slower than traditional), and RBAC at $0.4\times$. The compilation advantage emerges as k grows beyond 50, where indexed evaluation increasingly outperforms linear scanning.

ABAC shows the largest performance difference. U-HAP’s latency grows from $14.6\ \mu\text{s}$ at $k=10$ to $32.1\ \mu\text{s}$ at $k=500$ —a $2.2\times$ increase for a $50\times$ increase in rules. The traditional evaluator grows from $27.5\ \mu\text{s}$ to $1,392.0\ \mu\text{s}$ ($50.6\times$ increase), consistent with $O(k \cdot g)$ complexity. The resulting speedup grows from $1.9\times$ at $k=10$ to $43.4\times$ at $k=500$.

ACL demonstrates the benefit of hash-set membership checking. U-HAP’s ACL latency remains constant at approximately $0.7\text{--}0.8\ \mu\text{s}$ regardless of k , while the traditional evaluator’s linear scan grows from $0.6\ \mu\text{s}$ at $k=10$ to $15.9\ \mu\text{s}$ at $k=500$. At $k=10$, the traditional evaluator is marginally faster ($0.8\times$) because its scan cost is negligible; the crossover occurs near $k=50$ ($2.7\times$), and speedup reaches $20.3\times$ at $k=500$.

RBAC reveals that U-HAP’s bit-vector encoding incurs a construction overhead that exceeds the cost of simple linear scanning when ABAC evaluation is absent. U-HAP operates at $0.3\text{--}0.4\times$ of traditional speed across all k values. This overhead arises from encoding the user’s role set into an integer bit-vector and performing a bitwise AND with the precompiled role mask—operations whose Python-level cost exceeds a simple membership check when no ABAC predicates amortize the overhead.

Analysis. The per-model decomposition reveals that U-HAP’s overall performance advantage is primarily driven by ABAC hash consing, which achieves sub-linear scaling through memoized DAG evaluation. For $k=500$ ABAC rules with 10 gates each, the traditional evaluator performs up to 5,500 gate evaluations per request, while U-HAP evaluates each unique sub-expression at most once. ACL hash-set lookup provides a secondary benefit that grows with k . The RBAC result represents an honest trade-off: in mixed-model workloads containing RBAC, ABAC, ACL, and deny rules together, the RBAC overhead is absorbed by the ABAC and ACL savings, yielding the $3.0\text{--}3.2\times$ net speedup observed in Section IV-A (Fig. 2). For deployments consisting exclusively of simple RBAC rules without ABAC predicates, traditional sequential evaluation remains more efficient.

D. Summary of Results

Table II consolidates all experimental results. The p95 column confirms that tail latency remains close to the median, indicating low variance across all 1,000 timed iterations.

TABLE II
SUMMARY OF EXPERIMENTAL RESULTS (MEDIAN AND P95 LATENCY).

Experiment	Config	U-HAP (μs)	p95 (μs)	Trad. (μs)	Speedup
Namespace ($N=1,000$)	Cache OFF	18.0	18.5	56.8	$3.2\times$
	Cache ON	3.6	3.6	–	$15.9\times$
ABAC ($k=500$)	–	32.1	36.2	1,392.0	$43.4\times$
ACL ($k=500$)	–	0.8	0.8	15.9	$20.3\times$
RBAC ($k=500$)	–	49.0	49.3	13.6	$0.3\times$
Compilation (Phase 2)	46 rules	9.9 ms (one-time)			
	500 ABAC	187 ms (one-time)			

E. Threats to Validity

Internal validity. All measurements use worst-case probe requests where the matching rule is placed last. In production, early short-circuiting via deny or ACL matches would yield

lower latency, so reported speedups represent conservative lower bounds.

Construct validity. The baseline is a hand-written sequential evaluator, not a production system such as OPA or Kyverno. While this ensures a fair algorithmic comparison (same language, same process, no network), it does not capture deployment-specific factors such as serialization overhead or language-runtime optimizations in Go-based systems.

External validity. The policy configuration (46 rules, 10 gates per ABAC predicate, 50% atom sharing) is representative of medium-complexity deployments. Environments with substantially fewer rules ($k < 50$) may not benefit from compilation overhead, as shown by the $k=10$ data points in Fig. 3. Conversely, very large rule sets ($k > 500$) were not evaluated but are expected to amplify the speedup trend.

V. CONCLUSION

We presented U-HAP, a compilation-driven authorization framework that unifies RBAC, ABAC, ACL, and deny policies under a single graph-based semantic model for Kubernetes. By addressing the fragmentation of independently evaluated models, U-HAP shifts policy reasoning to compile time, pre-compiling policies into (n, r, a) -scoped artifacts and applying a universal graph abstraction for consistent semantics. Compilation is lightweight (9.9 ms for 46 rules), and the resulting indices are amortized over all subsequent requests. Our evaluation demonstrates: (i) $O(1)$ namespace-isolated lookup scaling to 1,000 namespaces with no latency degradation; (ii) $3.0\text{--}3.2\times$ algorithmic speedup over traditional sequential evaluation through compiled indices, hash-consed predicate DAGs, and bit-vector RBAC matching; (iii) up to $15.9\times$ speedup with decision caching for repeated requests; and (iv) $43.4\times$ speedup for ABAC-intensive workloads at $k=500$ rules, driven by hash-consed sub-expression memoization that reduces evaluation from $O(k \cdot g)$ to $O(\text{unique DAG nodes})$. The compilation advantage emerges at $k \geq 50$; for smaller rule sets, the indexing overhead exceeds the benefit.

The current implementation has several limitations: (i) policies are scoped to individual namespaces, without cross-namespace or ClusterRole support; (ii) evaluation is single-threaded per request; (iii) for workloads consisting exclusively of simple RBAC rules without ABAC predicates, U-HAP’s bit-vector construction overhead results in slower performance compared to sequential evaluation; and (iv) baseline comparison uses an in-process sequential evaluator rather than a production deployment, which, while ensuring a fair algorithmic comparison, does not capture deployment-specific overheads. Future work includes extending U-HAP to cluster-wide policies, integrating with production authorization systems for end-to-end benchmarking, supporting dynamic policy hot-reload without service restart, and exploring batch evaluation for high-throughput multi-tenant environments.

REFERENCES

- [1] NSA and CISA, “Kubernetes Hardening Guidance,” NSA Cybersec. Tech. Report, ver. 1.2, Aug. 2022. [Online]. Avail-

able: https://media.defense.gov/2022/Aug/29/2003066362/-1/-1/0/CTR_KUBERNETES_HARDENING_GUIDANCE_1.2_20220829.PDF

- [2] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security Misconfigurations in Open Source Kubernetes Manifests: An Empirical Study," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 4, 2023.
- [3] N. Yang, W. Shen, J. Li, X. Liu, X. Guo, and J. Ma, "Take Over the Whole Cluster: Attacking Kubernetes via Excessive Permissions of Third-Party Applications," in *Proc. ACM CCS*, 2023, pp. 3048–3062.
- [4] G. Rostami, "Role-Based Access Control (RBAC) Authorization in Kubernetes," *J. ICT Standardization*, vol. 11, no. 3, pp. 237–260, 2023, doi: 10.13052/jicts2245-800X.1132.
- [5] N. Farhadighalati, L. A. Estrada-Jimenez, S. Nikghadam-Hojjati, and J. Barata, "A Systematic Review of Access Control Models: Background, Existing Research, and Challenges," *IEEE Access*, vol. 13, pp. 17777–17806, 2025, doi: 10.1109/ACCESS.2025.3533145.
- [6] M. S. Rahaman, S. N. Tisha, E. Song, and T. Cerny, "Access Control Design Practice and Solutions in Cloud-Native Architecture: A Systematic Mapping Study," *Sensors*, vol. 23, no. 7, art. no. 3413, 2023.
- [7] Z. Morić, V. Dakić, and T. Čavala, "Security Hardening and Compliance Assessment of Kubernetes Control Plane and Workloads," *J. Cybersec. Privacy*, vol. 5, no. 2, art. no. 30, 2025.
- [8] OASIS, "eXtensible Access Control Markup Language (XACML) Version 3.0 Plus Errata 01," OASIS Standard, 2017.
- [9] M. Yang, V. Atluri, S. Sural, and J. Vaidya, "A Graph-Based Framework for ABAC Policy Enforcement and Analysis," in *Proc. DBSec*, 2024, pp. 3–23.
- [10] Y. Gu, X. Tan, Y. Zhang, S. Gao, and M. Yang, "EPScan: Automated Detection of Excessive RBAC Permissions in Kubernetes Applications," in *Proc. 2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 3199–3217, doi: 10.1109/SP61157.2025.00011.
- [11] R. Chandramouli and Z. Butcher, "A Zero Trust Architecture Model for Access Control in Cloud-Native Applications in Multi-Location Environments," NIST SP 800-207A, 2023.
- [12] H. Nguyen *et al.*, "PerfSPEC: Performance Profiling-Based Proactive Security Policy Enforcement for Containers," *IEEE Computer*, 2025, doi: 10.1109/MC.2025.10577533.
- [13] L. Ma *et al.*, "Research on Authorization Model of Attribute Access Control Based on Knowledge Graph," in *Proc. UbiSec*, Springer, 2024, pp. 350–364.
- [14] R. Pang, R. Caceres, M. Burrows, *et al.*, "Zanzibar: Google's Consistent, Global Authorization System," in *Proc. USENIX ATC*, 2019, pp. 33–46.
- [15] M. Davari and M. Zulkernine, "Automatic Conversion of ABAC Policies for RBAC Systems," in *Proc. 2023 IEEE Conference on Dependable and Secure Computing (DSC)*, 2023, pp. 1–7, doi: 10.1109/DSC61021.2023.10354157.
- [16] A. Sissodiya, E. Chiquito, U. Bodin, and J. Kristiansson, "Formal Verification for Preventing Misconfigured Access Policies in Kubernetes Clusters," *IEEE Access*, vol. 13, pp. 141798–141813, 2025, doi: 10.1109/ACCESS.2025.3597504.
- [17] S. Kern, T. Baumer, S. Groll, L. Fuchs, and G. Pernul, "Optimization of Access Control Policies," *J. Inf. Secur. Appl.*, vol. 70, art. no. 103301, 2022, doi: 10.1016/j.jisa.2022.103301.
- [18] CNCF, "Open Policy Agent," 2024. [Online]. Available: <https://www.openpolicyagent.org> (accessed: Mar. 25, 2026).
- [19] Kyverno Project, "Kyverno: Kubernetes Native Policy Management," 2024. [Online]. Available: <https://kyverno.io> (accessed: Mar. 25, 2026).
- [20] A. X. Liu, F. Chen, J. Hwang, and T. Xie, "Designing Fast and Scalable XACML Policy Evaluation Engines," *IEEE Trans. Computers*, vol. 60, no. 12, pp. 1802–1817, Dec. 2011, doi: 10.1109/TC.2011.15.